model and task, the model generally should be able to perform the task well (and perform well on counterfactuals). Performance for all models and tasks using 0-shot prompts[5] and greedy decoding is in Table 1; performance on counterfactual inputs are in App. C.5. Gemma 2 and Llama are generally capable across the board, performing well on MCQA and ARC (Easy). GPT-2 Small has been extensively studied in MI; it is significantly smaller than the other models we investigate and less capable (and thus may rely on qualitatively different mechanisms than larger models that have been exposed to more data). Qwen-2.5 performs well relative to its size but has not yet been extensively studied.

### 2.4. Leaderboard

We have constructed two online leaderboards (one for each track) hosted on Huggingface to receive user submissions and display results. We intend for the leaderboards to serve as a living public artifact that both incentivizes progress in MI and advises users of MI tools on the state of the art. More details about the leaderboards, including screenshots, are in App. C.6.

We have constructed a submission portal for users. This will aggregate required information and links to one's materials, where they will then be used to run evaluations on the private test set. Our hope is that users will be able to use the public test set for fast prototyping of new methods, and that the private test set will be the measure by which the state of the art is benchmarked.

## 3. Circuit Localization Track

The circuit localization track centers on evaluating how well a method can discover causal subgraphs $\mathcal{C}$—more commonly known as **circuits** (Olah et al., 2020)—that localize the mechanisms underlying how a full neural network $\mathcal{N}$ performs a given task. Here, we define metrics for comparing circuit localization methods (§3.1) and compare common methods (§3.2).

**Defining circuits.** A circuit $\mathcal{C}$ is a set of nodes and edges in the computation graph of $\mathcal{N}$. Nodes are typically submodules or attention heads (e.g., the layer 5 MLP, or attention head 10 at layer 12); edges are abstract objects that reflect information flow between a pair of nodes.

### 3.1. Circuit Metrics

The metrics used to evaluate circuits are largely not standardized. There have been efforts to bring rigor to circuit evaluations (Miller et al., 2024)—and to ascertain whether circuits are sensible data structures at all (Shi et al., 2024). As far as we are aware, no large-scale benchmark exists to systematically compare circuit localization methods. Thus, in addition to the proposed range of models and tasks (§2), we propose two circuit localization metrics that enable comparison across circuit discovery *methods*, rather than across individual circuits.

A common metric for evaluating circuits is **faithfulness** (Wang et al., 2023; Miller et al., 2024). It aims to measure the extent to which a subgraph $\mathcal{C}$ of the computation graph explains the full model $\mathcal{N}$'s behavior on some task. Faithfulness is often defined ad hoc. In fact, this metric is often used for two distinct goals: (i) to measure whether $\mathcal{C}$ contributes to higher performance on a task (e.g., Meng et al., 2022; Stolfo et al., 2023; Nikankin et al., 2025), or (ii) to capture *any* component with measurable impact on a task, whether positive or negative (e.g., Wang et al., 2023; Hanna et al., 2024; Marks et al., 2025). Which is the correct way to measure the quality of a circuit? Many studies work with either one of the notions, or a mix of the two—e.g., discovering components as in (ii) but defining the metric more in line with (i), or vice versa. This overloads the term.

We claim that both are valid but complementary goals, and therefore split faithfulness into two metrics: (i) the **integrated circuit performance ratio** (CPR), and (ii) the **integrated circuit-model distance** (CMD). CPR prioritizes methods that locate components with a positive effect on model performance on the task; higher is better. CMD prioritizes methods that locate components with *any* strong effect on model performance, including negative effects; 0 is best. Intuitively, CPR may be more useful when aiming to understand components that encourage or positively affect a given model behavior. CMD may be more useful when the aim is to explain the full algorithm the model implements to perform some behavior (including cases where the behavior is not desirable). We operationalize these metrics below.

Discovering a circuit $\mathcal{C}$ often involves scoring components in a computation graph according to their importance, and only including those that exceed a causal importance threshold $\lambda$ (a hyperparameter; Conmy et al., 2023; Marks et al., 2025). Given $\mathcal{C}$ and the full neural network $\mathcal{N}$, we can define **faithfulness** $f$ as

$$f(\mathcal{C}, \mathcal{N}; m) = \frac{m(\mathcal{C}) - m(\varnothing)}{m(\mathcal{N}) - m(\varnothing)}, \tag{1}$$

where $m$ is the logit difference $y' - y$ between the correct answer $y$ given the original input $x$ and correct answer $y'$ given the counterfactual input $x'$.[6] $\varnothing$ is the model with *all* components ablated (the empty circuit). Conceptually, this

[5] In-context learning is itself a behavior worth studying mechanistically, but one that is challenging to disentangle from task-specific behavior, leading many MI studies to use 0-shot prompts.

[6] No choice of $m$ is perfect. Like the counterfactual input, the metric defines the task. We follow Zhang & Nanda (2024), who recommend the logit difference based on empirical evidence.

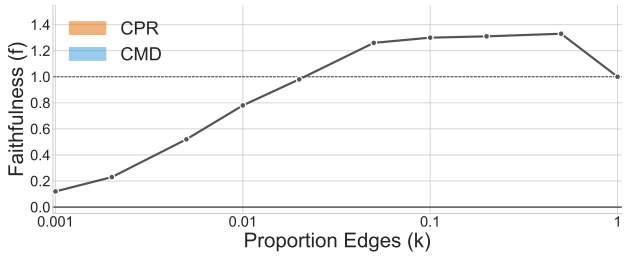


*Figure 2.* Definition of our faithfulness metrics. CPR, in orange, is the area under the faithfulness curve (the **black** line); it captures how well the method finds performant circuits at many circuit sizes. CMD, in blue, is the area between the faithfulness curve and the line at $f = 1$; it captures how closely the circuit's behavior resembles the model's task-specific behavior at many circuit sizes. Because we define $f$ as a ratio, $f = 1$ (the horizontal line) means that the circuit and full model achieve the same logit difference.

corresponds to the proportion of divergence in $m$ between the prior and the full model that the circuit recovers (Marks et al., 2025). There exist other formulations of $f$, like the *difference* (rather than *ratio*) of $m$ between $\mathcal{C}$ and $\mathcal{N}$ (Wang et al., 2023). We opt for the formulation of Marks et al. (2025), as this gives meaning to the values 0 ($\mathcal{C}$ recovers none of the performance of $\mathcal{N}$ relative to $\varnothing$) and 1 ($\mathcal{C}$ assigns identical probability differences to $y$ and $y'$ relative to $\mathcal{N}$).

We do not want the threshold $\lambda$ to affect comparisons between circuit localization methods. Thus, we propose shifting focus away from the quality of individual circuits, and toward a method's Pareto optimality with respect to two criteria: localizing task behavior, and minimizing circuit size. This formulation has the benefit of capturing **minimality** and **faithfulness** simultaneously.[7] Specifically, we propose to to quantify CPR as the area under the faithfulness curve with respect to circuit size, and to quantify CMD as the area between the faithfulness curve and 1. Both metrics involve evaluating faithfulness at many circuit sizes, and can be conceptualized as marginalizing over the circuit size hyperparameter.

In their exact form, $\text{CPR} = \int_{k=0}^{1} f(\mathcal{C}_k)\, dk$ and $\text{CMD} = \int_{k=0}^{1} |1 - f(\mathcal{C}_k)|\, dk$, where $f$ is the faithfulness of circuit $\mathcal{C}_k$ and $k$ is the proportion of edges from $\mathcal{N}$ in $\mathcal{C}_k$. Computing these integrals would require infinite samples. Instead, we measure faithfulness at a few representative circuit sizes, and use these to approximate CPR and CMD with a Riemann sum:

1. For all proportions of components $k \in \{.001, .002, .005, .01, .02, .05, .1, .2, .5, 1\}$, discover a circuit $\mathcal{C}_k$ such that $\frac{|\mathcal{C}_k|}{|\mathcal{N}|} \leq k$.
2. Compute the faithfulness $f$ for all $\mathcal{C}_k$.
3. Compute the area under $f$ (CPR) and the area between $f$ and 1 (CMD) using the trapezoidal rule.

We illustrate CPR and CMD in Figure 2.

In a realistic neural network, it is difficult to anticipate the best-case and worst-case CPR or CMD values, meaning we cannot bound the metric without losing information. If we knew the ground-truth circuit, we could instead compute precision and recall. Thus, inspired by InterpBench (Gupta et al., 2024), we train a model that implements a known ground-truth circuit for IOI. Because we know which edges are in the circuit, we report AUROC ($\in [0, 1]$) over edges. See App. E for InterpBench model training and implementation details.

**Measuring circuit size.** Circuits can be defined at many levels of granularity: one can include entire layers, submodules in a layer, neurons in a submodule, etc. One can also define a circuit at the level of nodes (e.g., submodules) or edges (e.g., connections between submodules). Thus, a key challenge is defining a notion of circuit size that enables comparison across different types of circuits. For this purpose, we treat including a node as equivalent to including all of its outgoing edges. Including one neuron[8] of $d_{\text{model}}$ in submodule $u$ can be conceptualized as including all outgoing edges from $u$ to $\frac{1}{d_{\text{model}}}$ of the degree they would have been compared to including all neurons in $u$. Under these assumptions, we define the **weighted edge count**:

$$|\mathcal{C}| = \sum_{(u,v)\in\mathcal{C}} \left( \frac{|N_u \cap N_{\mathcal{C}}|}{|N_u|} \right), \tag{2}$$

where $u$ and $v$ are nodes (submodules), $N_u$ is the set of neurons in $u$ (the size of which is typically $d_{\text{model}}$), and $N_{\mathcal{C}}$ is the set of neurons in the circuit. Intuitively, this is the number of edges from a submodule weighted by the proportion of neurons from that submodule in the circuit; we sum this quantity over all submodules. We then normalize this count by the number of possible edges to obtain a percentage.

### 3.2. Circuit Localization Baselines

Here, we evaluate common circuit localization methods. We evaluate each model[9] and method[10] in §2 where possible. We compare across multiple axes of variation, including (a) circuit localization method (see below), (b) granularity,

[7] Prior implementations of minimality require manual analysis of the circuit (Wang et al., 2023); our formulation is more general, though it is useful primarily as a relative comparison point, rather than an absolute measure.

[8] We use "neuron" to refer to a single dimension of any hidden vector, regardless of whether it is preceded by a non-linearity.

[9] We do not evaluate Gemma or Qwen on Arithmetic, as they tokenize numbers such that each digit has its own token.

[10] Exact activation patching and optimal ablations become intractable with respect to runtime as models scale. UGS becomes intractable with respect to memory requirements as models scale.

*Table 2.* CMD scores across circuit localization methods and ablation types (lower is better), and AUROC scores for InterpBench (higher is better). All evaluations were performed using counterfactual ablations. Arithmetic scores are averaged across addition and subtraction; see Table 17 (App. D.3) for separate scores. We **bold** and underline the best and second-best methods per column, respectively.

| | | IOI | | | | Arithmetic | MCQA | | | ARC (E) | | ARC (C) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Method** | InterpBench (↑) | GPT-2 | Qwen-2.5 | Gemma-2 | Llama-3.1 | Llama-3.1 | Qwen-2.5 | Gemma-2 | Llama-3.1 | Gemma-2 | Llama-3.1 | Llama-3.1 |
| Random | 0.44 | 0.75 | 0.72 | 0.69 | 0.74 | 0.75 | 0.73 | 0.68 | 0.74 | 0.68 | 0.74 | 0.74 |
| EActP (CF) | 0.28 | **0.02** | 0.49 | - | - | - | 0.36 | - | - | - | - | - |
| EAP (mean) | 0.78 | 0.29 | 0.18 | 0.25 | 0.04 | 0.07 | 0.21 | 0.20 | 0.16 | 0.22 | 0.28 | 0.20 |
| EAP (CF) | 0.73 | 0.03 | 0.15 | 0.06 | **0.01** | 0.01 | 0.07 | 0.08 | **0.09** | **0.04** | **0.11** | **0.18** |
| EAP (OA) | 0.77 | 0.30 | 0.16 | - | - | - | 0.11 | - | - | - | - | - |
| EAP-IG-inp. (CF) | 0.71 | 0.03 | 0.02 | 0.04 | **0.01** | **0.00** | 0.08 | **0.06** | 0.14 | **0.04** | **0.11** | 0.22 |
| EAP-IG-act. (CF) | **0.81** | 0.03 | **0.01** | **0.03** | **0.01** | **0.00** | **0.05** | 0.07 | 0.13 | **0.04** | 0.30 | 0.37 |
| NAP (CF) | 0.30 | 0.38 | 0.33 | 0.37 | 0.29 | 0.28 | 0.30 | 0.35 | 0.32 | 0.33 | 0.69 | 0.69 |
| NAP-IG (CF) | 0.62 | 0.27 | 0.20 | 0.26 | 0.19 | 0.18 | 0.18 | 0.29 | 0.33 | 0.28 | 0.67 | 0.67 |
| IFR | 0.71 | 0.42 | 0.69 | 0.75 | 0.83 | 0.22 | 0.60 | 0.62 | 0.48 | 0.66 | 0.64 | 0.76 |
| UGS | 0.74 | 0.03 | 0.03 | - | - | - | 0.20 | - | - | - | - | - |

including edge-level and neuron-level circuits, and (c) ablation type, including counterfactual (CF) ablations, mean ablations, and "optimal ablations" (OA; Li & Janson, 2024). For all methods, we assign importance scores to all edges or nodes, and either include the top-scoring components or perform greedy search; see App. D.1 for details.

As a sanity check, we compare to random control circuits (RANDOM). We operationalize this by uniformly sampling an importance score in $[-1, 1]$ for all edges in the model. We take the mean CPR and CMD across 3 random seeds.

One way to find circuits is to filter model components by their indirect effects (IE; Pearl, 2001). The IE is defined as the change in $m$ caused by replacing a component's activation with its activation on another input, typically one where the expected output differs. We follow the procedure of Vig et al. (2020) and (Finlayson et al., 2021). However, computing IE in exact form (ACTIVATION PATCHING, or ActP) is expensive, requiring $O(n)$ forward passes, where $n$ is the number of possible edges in $\mathcal{N}$.

**Attribution methods** aim to reduce the cost of computing IE by approximating it. Nanda (2023) and Syed et al. (2024) propose ATTRIBUTION PATCHING (AP), which linearly approximates the IE for all nodes or edges in $O(1)$ forward passes. When performed at the node level, we call this NODE AP (NAP); at the edge level, it is EDGE AP (EAP).

Unfortunately, AP approximates IE poorly (Syed et al., 2024). AP WITH INTEGRATED GRADIENTS (Sundararajan et al., 2017) or AP-IG, improves on AP by performing multiple steps of AP, trading off speed for approximation quality. We test two AP-IG variants: (i) AP-IG-inputs (Hanna et al., 2024) and (ii) AP-IG-activations (Marks et al., 2025). AP-IG-inputs requires $O(Z)$ forward passes, while AP-IG-activations requires $O(Z \cdot L)$, where $Z$ is the number of AP steps, and $L$ is the number of model layers. We use $Z = 5$, following Hanna et al. (2024). We explore AP-IG at the node level (NAP-IG) and edge level (EAP-IG). See App. D.2 for definitions and details.

**Information flow routes** (IFR; Ferrando & Voita, 2024) is a non-counterfactual-based and non-causal method that includes edge $(u, v)$ in $\mathcal{C}$ if the output vector of $u$ and input vector of $v$ are highly similar; this is taken as evidence of a writing operation. See App. D.2 for details on how we adapted IFR to our formalization of circuits.

**Mask-based methods** aim to learn a pruning mask on the edges of the computational graph. During training, masks are continuous. Edges with low mask values are ablated more often or more fully; edges with high values are likely in the circuit. The mask's training objective aims to maintain model behavior (often measured by KL-divergence with the unablated model) while keeping the mask sparse (measured by its $L_1$ norm). After training, the mask can be converted into a binary mask indicating which edges are in the circuit.

As a mask-based method, we employ UNIFORM GRADIENT SAMPLING (UGS; Li & Janson, 2024), which uses CF ablations. Li & Janson also propose optimal ablations (OA), in which the mask is learned jointly with the value used to ablate non-circuit edges; they prefer OA as it is both independent of the example being ablated (unlike CF ablations) and minimally harmful (unlike mean ablations). Due to computational constraints, we do not do this, but instead learn OA vectors by optimizing them to minimize the expected cross-entropy loss on the task dataset. We then use these vectors as ablation values when running circuit localization with EAP. See App. D.2 for details.

### 3.3. Results

We present CMD and AUROC scores (Table 2) for each method and task where it is tractable to run; see Table 14 in App. D for CPR scores. On both metrics, EAP-IG-inputs with CF ablations generally performs best. EAP-IG-